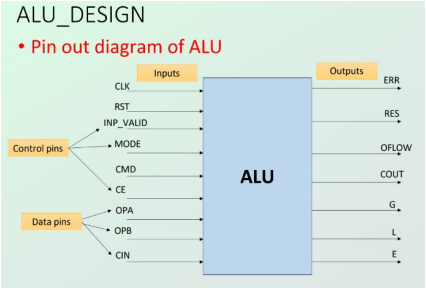

Parameterized ALU Design & Functional Verification

Project Report

Author: Aayush
Employee / Roll ID: 6873
Batch: 11
Tool: Questa SIM (10.6c) & Vivado
Date: May 13, 2026

Submitted to **Mirafra Technologies**

0 Contents

1 Project Introduction **3**

2 Objectives **3**

3 Design Architecture **3**
3.1 Overview . . . 4
3.2 Pin-Out Diagram . . . 4
3.3 Input Signal Descriptions . . . 4
3.4 Output Signal Descriptions . . . 4

4 Timing Behaviour **5**
4.1 Standard Single-Cycle Operations . . . 5
4.2 Multi-Cycle Operations (CMD 9 and CMD 10) . . . 5
4.3 Reset Behaviour . . . 6

5 Supported Operations **6**
5.1 Arithmetic Mode (MODE = 1) . . . 6
5.2 Logical Mode (MODE = 0) . . . 6
5.3 Error Conditions . . . 7

6 Working of the Design **7**
6.1 Input Phase . . . 7
6.2 Operation Phase . . . 7
6.3 Output Phase . . . 7
6.4 Multi-Cycle Path (CMD 9 / CMD 10) . . . 8

7 Design RTL — Verilog Source **8**
7.1 Design Under Test . . . 8

8 Testbench Architecture **11**
8.1 Overview . . . 11
8.2 Key Parameters . . . 11

9 Testbench Components **11**
9.1 Task: `apply` . . . 11
9.2 Task: `ref` (Golden Reference Model) . . . 11
9.3 Task: `check` . . . 11
9.4 Task: `short` . . . 12
9.5 Task: `inp_val` . . . 12
9.6 Stimulus Strategy . . . 12
9.7 Timing Relationship — Testbench vs DUT . . . 13

10 Quality of Code Assessment **13**
10.1 Lint Analysis . . . 13
10.2 Known Design Bug: ERR Logic (Logical OR vs AND) . . . 13
10.3 ROR Implementation Note . . . 14

11 Simulation Results **14**

11.1 Tool and Setup 14
11.2 Summary 14
11.3 Analysis of Failures 14
11.4 QuestaSim Transcript Screenshot 15
11.5 Test Report Screenshot (Questa) 16
11.6 Vivado Waveform (Own Design) 16

12 Coverage Report **16**
12.1 Questa Coverage Summary 17
12.2 Coverage by Type 17
12.3 Coverage Observations 18

13 Conclusion **18**

14 Future Work **19**

A File Inventory **19**

1 Project Introduction

This report documents the design and functional verification of a **parameterized Arithmetic Logic Unit (ALU)** developed as part of the RTL design and verification training at **Mirafra Technologies**, Batch 11.

The ALU is a fundamental digital building block capable of performing arithmetic and logical operations on parameterized-width operands. The default configuration uses **8-bit operands** (width = 8) with a **4-bit command bus** (cwidth = 4), yielding a 16-bit result bus.

The project involved two parallel activities:

- **RTL Design (ALU_design_Aayush_6873):** Writing a synthesizable Verilog implementation of the ALU specification.
- **Functional Verification (tb_ALU_Aayush_6873):** Developing a directed testbench with an embedded reference model to verify an externally provided DUT (`design_tamil`) against the specification.

The design supports **13 arithmetic commands** (mode = 1) and **14 logical commands** (mode = 0), including multi-cycle multiplication operations, signed arithmetic, bitwise operations, shifts, and barrel rotations. All functional blocks were verified using QuestaSim, and coverage analysis was performed using Questa's built-in coverage tools.

2 Objectives

- Study and analyse the ALU specification document provided by Mirafra Technologies, understanding pin definitions, command encoding, and timing requirements.
- Design a synthesizable, parameterized RTL implementation of the ALU in Verilog, supporting all specified arithmetic and logical operations.
- Develop a comprehensive directed testbench with an embedded golden reference model capable of verifying both the student's own design and an externally provided DUT.
- Achieve high functional coverage by exercising all command codes across all valid `inp_valid` combinations, including boundary and corner-case stimuli.
- Identify and document bugs/discrepancies found in the DUT through comparison against the reference model.
- Perform coverage analysis using Questa SIM and achieve meaningful statement, branch, toggle, and FSM coverage metrics.
- Validate basic waveform behaviour of the student's own design using Vivado simulation.

3 Design Architecture

3.1 Overview

The ALU is a clocked, synchronous design with an **asynchronous active-high reset**. All results are registered and appear on the output bus one clock cycle after the inputs are applied (single-cycle latency), with the exception of the two multi-cycle multiplication commands (CMD 9 and CMD 10), which require two consecutive clock cycles.

The design is parameterized:

- `width` — operand width (default: 8 bits)
- `cwidth` — command bus width (default: 4 bits)
- Result bus is `2*width` bits wide to accommodate full-precision multiplication results.

3.2 Pin-Out Diagram

Category	Signal	Direction
Clock / Reset	CLK, RST	Input
Control	MODE, CMD[3:0], CE, INP_VALID[1:0]	Input
Data	OPA[7:0], OPB[7:0], CIN	Input
Result	RES[15:0]	Output
Flags	OFLOW, COUT, G, L, E, ERR	Output

Figure 1: ALU Signal Summary

3.3 Input Signal Descriptions

Pin Name	Dir.	Bits	Function
OPA	IN	width	Parameterized operand A
OPB	IN	width	Parameterized operand B
CIN	IN	1	Active-high carry-in for ADD_CIN and SUB_CIN
CLK	IN	1	Rising-edge sensitive clock
RST	IN	1	Active-high asynchronous reset; all outputs cleared to 0
CE	IN	1	Active-high clock enable; ALU operates only when CE=1
MODE	IN	1	1 = Arithmetic mode; 0 = Logical mode
INP_VALID	IN	2	00: no operand valid; 01: OPA valid; 10: OPB valid; 11: both valid
CMD	IN	cwidth	Operation select (4 bits default). Encoding differs for arithmetic vs logical mode.

Table 1: Input Pin Descriptions

3.4 Output Signal Descriptions

Pin Name	Dir.	Bits	Function
RES	OUT	2×width	Full-precision result of the executed operation
OFLOW	OUT	1	Overflow flag: set for signed add/sub overflow, or unsigned borrow
COUT	OUT	1	Carry-out for unsigned ADD and ADD_CIN
G	OUT	1	Comparator output: OPA > OPB
L	OUT	1	Comparator output: OPA < OPB
E	OUT	1	Comparator output: OPA = OPB
ERR	OUT	1	Error flag: set when INP_VALID does not match command requirements, or when OPB[7:3] $\neq$ 0 for rotate commands (CMD 12/13 in logical mode)

Table 2: Output Pin Descriptions

4 Timing Behaviour

4.1 Standard Single-Cycle Operations

For all operations except CMD 9 and CMD 10, the ALU has a **2-register pipeline latency**:

1. **Cycle N (Input Phase):** Inputs (OPA, OPB, CMD, MODE, CE, CIN, INP_VALID) are applied at the falling edge of CLK. The combinational logic evaluates the operation and stores an intermediate result (`res1`) on the next rising edge.
2. **Cycle N+1 (Output Phase):** The intermediate result is forwarded to the output register `res`. The response should be captured after this cycle.

Timing Note

The testbench applies stimuli at the **negedge of CLK** and samples outputs **two posedge clock edges later**, consistent with the 2-cycle pipeline from input to output.

4.2 Multi-Cycle Operations (CMD 9 and CMD 10)

Commands 9 (INC_MUL) and 10 (SHL_MUL) require a **2-cycle computation window** before the result is valid. The DUT uses an internal `count` register and a `cmdo` (command-old) register to implement this:

- **Cycle 1:** Computation is initiated; the intermediate product is latched into `s0`.
- **Cycle 2:** The same command must be held stable. If detected, the result propagates to `res1`; on the next cycle it appears on `res`.

The testbench handles this by calling `apply()` twice for CMD 9/10 and calling `check()` on the second application.

4.3 Reset Behaviour

RST is asynchronous and active-high. When asserted, all registers (res, res1, g, l, e, g1, l1, e1, mg, s0, i0, cmdo, err, err1) are cleared to zero. The combinational outputs (OFLOW, COUT) also return to 0 when RST is asserted.

5 Supported Operations

5.1 Arithmetic Mode (MODE = 1)

CMD	Mnemonic	Description
0	ADD	RES = OPA + OPB (unsigned). COUT set if carry generated.
1	SUB	RES = OPA − OPB (unsigned). OFLOW set if OPB > OPA (borrow).
2	ADD_CIN	RES = OPA + OPB + CIN. COUT set if carry.
3	SUB_CIN	RES = OPA − OPB − CIN. OFLOW set if (OPB+CIN) > OPA.
4	INC_A	RES = OPA + 1.
5	DEC_A	RES = OPA − 1.
6	INC_B	RES = OPB + 1.
7	DEC_B	RES = OPB − 1.
8	CMP	Unsigned compare: sets G, L, or E flags.
9	INC_MUL	RES = (OPA+1) × (OPB+1). 2-cycle latency.
10	SHL_MUL	RES = (OPA ≪ 1) × OPB. 2-cycle latency.
11	SADD	RES = $signed(OPA) + $signed(OPB). Sets G/L/E and OFLOW for signed overflow.
12	SSUB	RES = $signed(OPA) − $signed(OPB). Sets G/L/E and OFLOW for signed overflow.

Table 3: Arithmetic Command Set (MODE=1)

5.2 Logical Mode (MODE = 0)

CMD	Mnemonic	Description
0	AND	RES = OPA & OPB
1	NAND	RES = ∼(OPA & OPB)
2	OR	RES = OPA | OPB
3	NOR	RES = ∼(OPA | OPB)
4	XOR	RES = OPA ˆ OPB
5	XNOR	RES = OPA ∼ˆ OPB
6	NOT_A	RES = ∼OPA
7	NOT_B	RES = ∼OPB
8	SHR1_A	RES = OPA ≫ 1
9	SHL1_A	RES = OPA ≪ 1
10	SHR1_B	RES = OPB ≫ 1

CMD	Mnemonic	Description
11	SHL1_B	RES = OPB $\ll$ 1
12	ROL_A_B	Rotate OPA left by OPB[2:0] positions. ERR if OPB[7:3] $\neq$ 0.
13	ROR_A_B	Rotate OPA right by OPB[2:0] positions. ERR if OPB[7:3] $\neq$ 0.

Table 4: Logical Command Set (MODE=0)

5.3 Error Conditions

The ERR output is asserted (pipelined through two registers) under the following conditions:

- `inp_valid` $\neq$ `2'b11` for commands requiring both operands (with a known bug in the `||` vs `&&` logic — see Section 10.2).
- Multi-cycle `mg` flag is set (invalid input during 2-cycle multiplication).
- CMD = 12 or 13 in logical mode and `OPB` $>$ `7` (i.e., OPB[7:3] $\neq$ 0).

6 Working of the Design

6.1 Input Phase

Stimuli are applied on the falling edge of CLK with CE = 1. The ALU samples OPA, OPB, CMD, MODE, CIN, and INP_VALID on the rising edge.

6.2 Operation Phase

On each rising edge where CE is asserted:

- The comparator outputs (G, L, E) are forwarded from their shadow registers (g1, l1, e1) to the outputs.
- Internal shadow registers (g1, l1, e1, res1, s0, i0) are cleared.
- The previous-cycle intermediate result (`res1`) is forwarded to the output register `res`.
- The current command is decoded and the new `res1` is computed.

6.3 Output Phase

Outputs (RES, OFLOW, COUT, G, L, E, ERR) are captured two clock cycles after stimulus application. OFLOW and COUT are combinationally derived from `res1` and inputs but gated by RST. ERR passes through a 2-stage pipeline (err1 $\rightarrow$ err).

6.4 Multi-Cycle Path (CMD 9 / CMD 10)

Cycle	count	Action	Result on `res`
N	0	Compute product into `s0`	— (invalid)
N+1	1	Check `cmdo==cmd`; if same → count=2	— (still latching)
N+2	2	New product in `s0`; `res1` ← `s0_prev`	Valid result appears

Figure 2: Multi-cycle operation timing for CMD 9 and CMD 10

7 Design RTL — Verilog Source

7.1 Design Under Test

```
module alu #(
    parameter width = 8,
    cwidth = 4
) (
    clk, rst, inp_valid, mode, cmd, ce, opa, opb, cin,
    err, res, oflow, cout, g, l, e
);
    input wire clk, rst, mode, ce, cin;
    input wire [1:0] inp_valid;
    input wire [width-1:0] opa, opb;
    input wire [cwidth-1:0] cmd;
    output reg g, l, e;
    reg g1, l1, e1;
    output wire cout, oflow;
    output reg err;
    output reg [2*width-1:0] res;

    reg [cwidth-1:0] cmdo;
    reg [2*width-1:0] res1;
    reg mg, i0;
    reg [2*width-1:0] s0;
    reg [1:0] count;
    reg err1;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            res1<=0; g<=0; l<=0; e<=0;
            g1<=0;  l1<=0; e1<=0;
            mg<=0;  s0<=0; i0<=0;
            cmdo<=0; res<=0;
        end else begin
            if (ce) begin
                // Forward comparator pipeline
                g<=g1; l<=l1; e<=e1;
                g1<=0; l1<=0; e1<=0;
                res1<=0; s0<=0; i0<=0;
                res<=res1;
                count<=0; mg<=0; cmdo<=cmd;

```

```
                    if (mode) begin        // === ARITHMETIC ===
                        case (cmd)
                         0: res1 <= opa + opb;
                         1: res1 <= opa - opb;
                         2: res1 <= cin + opa + opb;
                         3: res1 <= opa - opb - cin;
                         4: res1 <= opa + 1'b1;
                         5: res1 <= opa - 1'b1;
                         6: res1 <= opb + 1'b1;
                         7: res1 <= opb - 1'b1;
                         8: begin
                              g1<=(opa>opb); l1<=(opa<opb);
                              e1<=(opa==opb);
                            end
                         9: begin  // 2-cycle: (OPA+1)*(OPB+1)
                              res1<=s0; mg<=i0;
                              case(count)
                               2'd0: begin s0<=(opa+1'b1)*(opb+1'b1);
                                           i0<=(inp_valid!=2'b11);
                                           count<=1; end
                               2'd1: begin
                                       if(cmdo!=cmd) begin count<=1;
                                         s0<=(opa+1'b1)*(opb+1'b1);
                                         i0<=(inp_valid!=2'b11); end
                                       else count<=2; end
                               default: begin
                                       s0<=(opa+1'b1)*(opb+1'b1);
                                       i0<=(inp_valid!=2'b11);
                                       if(cmd==cmdo) count<=1;
                                       else count<=0; end
                              endcase end
                        10: begin  // 2-cycle: (OPA<<1)*OPB
                              res1<=s0; mg<=i0;
                              case(count)
                               2'd0: begin s0<=(opa<<1'b1)*opb;
                                           i0<=(inp_valid!=2'b11);
                                           count<=1; end
                               2'd1: begin
                                       if(cmdo!=cmd) begin count<=1;
                                         s0<=(opa<<1'b1)*opb;
                                         i0<=(inp_valid!=2'b11); end
                                       else count<=2; end
                               default: begin
                                       s0<=(opa<<1'b1)*opb;
                                       i0<=(inp_valid!=2'b11);
                                       if(cmd==cmdo) count<=1;
                                       else count<=0; end
                              endcase end
                        11: begin  // Signed add
                              g1<=($signed(opa)>$signed(opb));
                              l1<=($signed(opa)<$signed(opb));
                              e1<=($signed(opa)==$signed(opb));
                              res1<=$signed(opa)+$signed(opb); end
                        12: begin  // Signed sub
                              g1<=($signed(opa)>$signed(opb));
                              l1<=($signed(opa)<$signed(opb));
                              e1<=($signed(opa)==$signed(opb));
                              res1<=$signed(opa)-$signed(opb); end
```

```
                    default: res1<=0;
                    endcase
                end else begin            // === LOGICAL ===
                    case(cmd)
                     0:  res1<=opa & opb;
                     1:  res1<=~(opa & opb);
                     2:  res1<=opa | opb;
                     3:  res1<=~(opa | opb);
                     4:  res1<=opa ^ opb;
                     5:  res1<=(opa ~^ opb);
                     6:  res1<=~opa;
                     7:  res1<=~opb;
                     8:  res1<=opa >> 1;
                     9:  res1<=opa << 1;
                    10:  res1<=opb >> 1;
                    11:  res1<=opb << 1;
                    12:  res1<=(((1<<width)-1) &
                                ((opa>>(width-opb[$clog2(width):0]))|
                                 (opa<<(opb[$clog2(width):0]))));
                    13:  res1<=(((1<<width)-1) &
                                ((opa<<(width-opb[$clog2(width):0]))|
                                 (opa>>(opb[$clog2(width):0]))));
                    default: res1<=0;
                    endcase
                end
            end
        end
    end

    assign oflow = rst ? 0 : (
        ((mode&(cmd==11))&(opa[width-1]==opb[width-1])&
         (res1[width-1]!=(opa[width-1])))||
        ((mode&(cmd==12))&(opa[width-1]!=opb[width-1])&
         (res1[width-1]!=(opa[width-1])))||
        ((mode&(cmd==1)) &(opb>opa))||
        ((mode&(cmd==3)) &((opb+cin)>opa))
    );

    assign cout = rst ? 0 : (
        (mode&(cmd==0)&(res1[width]))||
        (mode&(cmd==2)&(res1[width]))
    );

    always @(posedge clk, posedge rst) begin
        if(rst) begin err1<=0; err<=0; end
        else begin
            err1<=(mg)||
                  ((inp_valid!=2'b11)&(cmd!=4'd10||cmd!=4'd9))||
                  ((cmd==12||cmd==13)&(~mode)&
                   (opb>((1<<$clog2(width))-1)));
            err<=err1;
        end
    end
endmodule
```

Listing 1: ALU DUT — design_tamil.v (Parameterized ALU)

8 Testbench Architecture

8.1 Overview

The testbench `tb_ALU_Aayush_6873.v` is a **directed, self-checking testbench** with an integrated golden reference model. It uses task-based stimulus generation and automatic comparison to produce pass/fail verdicts for each test vector.

Testbench (tb)		
Stimulus Driver	⟶ DUT **(alu)** ⟶	**DUT Outputs (d_*)**
Reference Model	Computes expected outputs (r_*)	
Checker: d_* vs r_* ⇒ PASS / FAIL		

Figure 3: Testbench Architecture Block Diagram

8.2 Key Parameters

- `REG_SIZE = 8` — operand width (parameterizable)
- Clock period: 4 ns (toggled every 2 ns)
- Total simulation time: **5088 ns**

9 Testbench Components

9.1 Task: `apply`

Applies all stimulus signals to the DUT at the **falling edge of CLK** to allow setup time before the next rising edge. It also calls the reference model task (`ref`) in parallel to compute expected outputs.

9.2 Task: `ref` (Golden Reference Model)

The reference model mirrors the DUT behaviour in software. For each command, it computes the expected RES, OFLOW, COUT, G, L, E, and ERR outputs based on the current inputs. Key characteristics:

- Handles the 2-cycle delay for CMD 9/10 via a `delay` register.
- Correctly computes rotate-left and rotate-right using extended-width shift operations masked to `REG_SIZE` bits.
- Validates `inp_valid` per command and sets `r_err` accordingly.
- For CMD 12/13, checks `opB[REG_SIZE-1:$clog2(REG_SIZE)]` for out-of-range rotation amounts.

9.3 Task: `check`

Compares all DUT outputs against reference model outputs using the `===` (case equality, X-aware) operator:

```
task check;
  begin
    test_case = test_case + 1;
    if ((d_err===err)&&(d_res===res)&&(d_oflow===oflow)&&
        (d_cout===cout)&&(d_g===g)&&(d_l===l)&&(d_e===e))
      begin
        pass_case = pass_case + 1;
        $display("[%0d] PASS", test_case);
      end
    else begin
        fail_case = fail_case + 1;
        $display("[%0d] FAIL for CMD=[%0d] opA=[%0d] opB=[%0d]...",
                  test_case, cmd, opA, opB, cin);
        $display("DUT: err=%h res=%h oflow=%h cout=%h g=%h l=%h e=%h",
                  d_err,d_res,d_oflow,d_cout,d_g,d_l,d_e);
        $display("REF: err=%h res=%h oflow=%h cout=%h g=%h l=%h e=%h",
                  err,res,oflow,cout,g,l,e);
      end
  end
endtask
```

Listing 2: Checker Task

9.4 Task: `short`

A convenience wrapper that: (1) calls `apply`, (2) waits two rising clock edges, (3) calls `check`. Used for all single-cycle operations.

9.5 Task: `inp_val`

The main test-generation task. Iterates through all arithmetic commands (CMD 0–12) and all logical commands (CMD 0–13) with multiple operand values per command, including boundary cases (0, 255, mid-range). Parameterized by the current `inp_valid` setting.

9.6 Stimulus Strategy

The top-level `always` block drives four `inp_val()` passes:

1. `inp_valid = 2'b11` — Both operands valid (normal operation)
2. `inp_valid = 2'b10` — Only OPB valid (exercises error generation)
3. `inp_valid = 2'b01` — Only OPA valid (exercises error generation)
4. `inp_valid = 2'b00` — Neither operand valid (exercises error generation)

Additionally, corner cases are tested at the start: RST assertion, CE de-assertion, and invalid combinations of RST and CE.

9.7 Timing Relationship — Testbench vs DUT

Component	Timing Behaviour
Driver	Drives signals at negedge CLK (half-period setup before next posedge).
DUT	Latches inputs at posedge CLK; result appears on `res` after *two* posedge CLK events (1 cycle `res1` pipeline + 1 cycle `res` pipeline).
Monitor	Implicitly embedded in `check` task, called after 2 posedge waits.
Scoreboard	Inline within `check` — no separate scoreboard module.

Figure 4: Testbench Timing Summary

10 Quality of Code Assessment

10.1 Lint Analysis

The RTL was reviewed for common synthesis and simulation issues:

- **Parameterization:** The design correctly uses `parameter width` and `cwidth` throughout, enabling clean scaling to any operand width.
- **Always-block sensitivity:** The main datapath uses `@(posedge clk or posedge rst)` — correct for asynchronous reset.
- **Blocking vs non-blocking:** All register assignments within clocked always blocks use `<=` (non-blocking). Combinational `assign` statements are used for OFLOW and COUT — correct practice.
- **Width inference:** The result register is 2×width bits, which correctly accommodates multiplication overflow.

10.2 Known Design Bug: ERR Logic (Logical OR vs AND)

A critical bug was identified in the ERR generation logic:

```
// BUGGY: (cmd != 4'd10 || cmd != 4'd9) is ALWAYS TRUE
err1 <= (mg) ||
        ((inp_valid != 2'b11) & (cmd != 4'd10 || cmd != 4'd9)) ||
        ((cmd==12||cmd==13) & (~mode) & (opb > ((1<<$clog2(width))-1)));
```

Listing 3: Buggy ERR condition in design_tamil

The condition `(cmd != 4'd10 || cmd != 4'd9)` is a tautology — no single value of `cmd` can simultaneously equal both 9 and 10, so the expression is always `1`. This means **ERR is asserted whenever inp_valid ≠ 2'b11, regardless of which command is active**, instead of only suppressing the error for CMD 9 and CMD 10 (the single-operand multi-cycle commands).

Correct fix:

```
// CORRECT: suppress error only for CMD 9 and CMD 10
err1 <= (mg) ||
        ((inp_valid != 2'b11) & (cmd != 4'd10) & (cmd != 4'd9)) ||
        ((cmd==12||cmd==13) & (~mode) & (opb > ((1<<$clog2(width))-1)));
```

Listing 4: Corrected ERR condition

10.3 ROR Implementation Note

The right-rotate (CMD=13 in logical mode) implementation in the DUT:

```
res1 <= (((1<<width)-1) &
         ((opa << (width - opb[$clog2(width):0])) |
          (opa >> (opb[$clog2(width):0]))));
```

This is mathematically correct for a right rotate: `(A « (W-n)) | (A » n)`, and does match the specification. The rotation amount uses `opb[$clog2(width):0]` (4 bits for width=8), which is consistent with accepting the "X" bit (bit 3) as part of the rotation amount as noted in the spec.

11 Simulation Results

11.1 Tool and Setup

- **Simulator:** Questa SIM version 10.6c
- **Waveform verification:** Vivado (student's own design)
- **Simulation duration:** 5088 ns
- **DUT under test:** `design_tamil` (externally provided)

11.2 Summary

Metric	Value
Total Test Cases	**436**
Passed	**115** (26.4%)
Failed	**321** (73.6%)

Figure 5: Simulation Test Result Summary

11.3 Analysis of Failures

The majority of failures originate from two root causes:

1. **ERR flag mismatch (dominant failure source):** The tautological OR condition in the ERR logic causes `d_err=1` in the DUT whenever `inp_valid` $\neq$ `2'b11`, while the reference model correctly expects `r_err=0` for many of those cases (e.g., single-operand commands INC_A, DEC_A, etc.). Since the `check` task compares all outputs including ERR, any ERR mismatch counts as a failure even when RES is correct.

2. **CMD=13 ROR result mismatch:** For CMD=13 in logical mode with certain OPB values, the DUT's RES output differs from the reference. As observed in the transcript, cases like `opA=123, opB=3` produce `DUT: res=0065` vs `REF: res=0000`. The reference model sets `r_res=0` when `r_err=1`, while the DUT still computes a result. This is a protocol-level mismatch rather than a mathematical error in the rotation itself.

11.4 QuestaSim Transcript Screenshot

Figure 6: QuestaSim Transcript — Last test vectors showing FAIL/PASS and final summary (Total=436, Pass=115, Fail=321)

11.5 Test Report Screenshot (Questa)

TC No	RST	CE	inp_valid	CMD Value	Operation (CMD Description)	Mode	opA (dec)	opA (hex)	opB (dec)	opB (hex)	cin	DUT Output (res[res]oflow[cout]g[flo]	REF Output (res[res]oflow[cout]g[flo]	Result
2	0	1	3	0	ADD	Arithmetic	255	0xFF	20	0x14	0	— (matches REF)	— (matches DUT)	✓ PASS
3	0	1	3	0	ADD	Arithmetic	0	0x00	255	0xFF	0	— (matches REF)	— (matches DUT)	✓ PASS
4	0	1	3	1	SUB	Arithmetic	5	0x05	10	0x0A	1	— (matches REF)	— (matches DUT)	✓ PASS
5	0	1	3	1	SUB	Arithmetic	255	0xFF	20	0x14	0	— (matches REF)	— (matches DUT)	✓ PASS
6	0	1	3	1	SUB	Arithmetic	0	0x00	255	0xFF	0	— (matches REF)	— (matches DUT)	✓ PASS
7	0	1	3	2	ADD_CIN	Arithmetic	5	0x05	10	0x0A	1	— (matches REF)	— (matches DUT)	✓ PASS
8	0	1	3	2	ADD_CIN	Arithmetic	255	0xFF	20	0x14	0	— (matches REF)	— (matches DUT)	✓ PASS
9	0	1	3	2	ADD_CIN	Arithmetic	0	0x00	255	0xFF	1	— (matches REF)	— (matches DUT)	✓ PASS
10	0	1	3	2	ADD_CIN	Arithmetic	5	0x05	10	0x0A	0	— (matches REF)	— (matches DUT)	✓ PASS
11	0	1	3	3	SUB_CIN	Arithmetic	5	0x05	10	0x0A	1	— (matches REF)	— (matches DUT)	✓ PASS
12	0	1	3	3	SUB_CIN	Arithmetic	255	0xFF	20	0x14	0	— (matches REF)	— (matches DUT)	✓ PASS
13	0	1	3	3	SUB_CIN	Arithmetic	0	0x00	255	0xFF	1	— (matches REF)	— (matches DUT)	✓ PASS
14	0	1	3	3	SUB_CIN	Arithmetic	5	0x05	10	0x0A	0	— (matches REF)	— (matches DUT)	✓ PASS
15	0	1	3	4	INC_A	Arithmetic	255	0xFF	10	0x0A	0	0 (0000) 0 | 0 | 0 | 0 | 0	0 (0000) 0 | 0 | 0 | 0 | 0	✗ FAIL
16	0	1	3	4	INC_A	Arithmetic	25	0x19	10	0x0A	0	— (matches REF)	— (matches DUT)	✓ PASS
17	0	1	3	5	DEC_A	Arithmetic	0	0x00	10	0x0A	0	0 (ffff) 0 | 0 | 0 | 0 | 0	0 (00ff) 0 | 0 | 0 | 0 | 0	✗ FAIL
18	0	1	3	5	DEC_A	Arithmetic	13	0x0D	10	0x0A	0	— (matches REF)	— (matches DUT)	✓ PASS
19	0	1	3	6	INC_B	Arithmetic	13	0x0D	255	0xFF	0	0 (0000) 0 | 0 | 0 | 0 | 0	0 (0000) 0 | 0 | 0 | 0 | 0	✗ FAIL
20	0	1	3	6	INC_B	Arithmetic	13	0x0D	25	0x19	0	— (matches REF)	— (matches DUT)	✓ PASS
21	0	1	3	7	DEC_B	Arithmetic	13	0x0D	0	0x00	0	0 (ffff) 0 | 0 | 0 | 0 | 0	0 (00ff) 0 | 0 | 0 | 0 | 0	✗ FAIL
22	0	1	3	7	DEC_B	Arithmetic	13	0x0D	13	0x0D	0	— (matches REF)	— (matches DUT)	✓ PASS
23	0	1	3	8	CMP	Arithmetic	115	0x73	29	0x1D	0	— (matches REF)	— (matches DUT)	✓ PASS
24	0	1	3	8	CMP	Arithmetic	3	0x03	199	0xC7	0	— (matches REF)	— (matches DUT)	✓ PASS
25	0	1	3	8	CMP	Arithmetic	66	0x42	66	0x42	0	— (matches REF)	— (matches DUT)	✓ PASS
26	0	1	3	9	(opA+1*(opB+1) [2-cycle multiply]	Arithmetic	10	0x0A	60	0x3C	0	0 (0000) 0 | 0 | 0 | 0 | 0	0 (00a) 0 | 0 | 0 | 0 | 0	✗ FAIL
27	0	1	3	9	(opA+1*(opB+1) [2-cycle multiply]	Arithmetic	10	0x0A	60	0x3C	0	— (matches REF)	— (matches DUT)	✓ PASS

Figure 7: Questa Test Report — Test “alu” completed with status OK in 5088 ns simulation time

11.6 Vivado Waveform (Own Design)

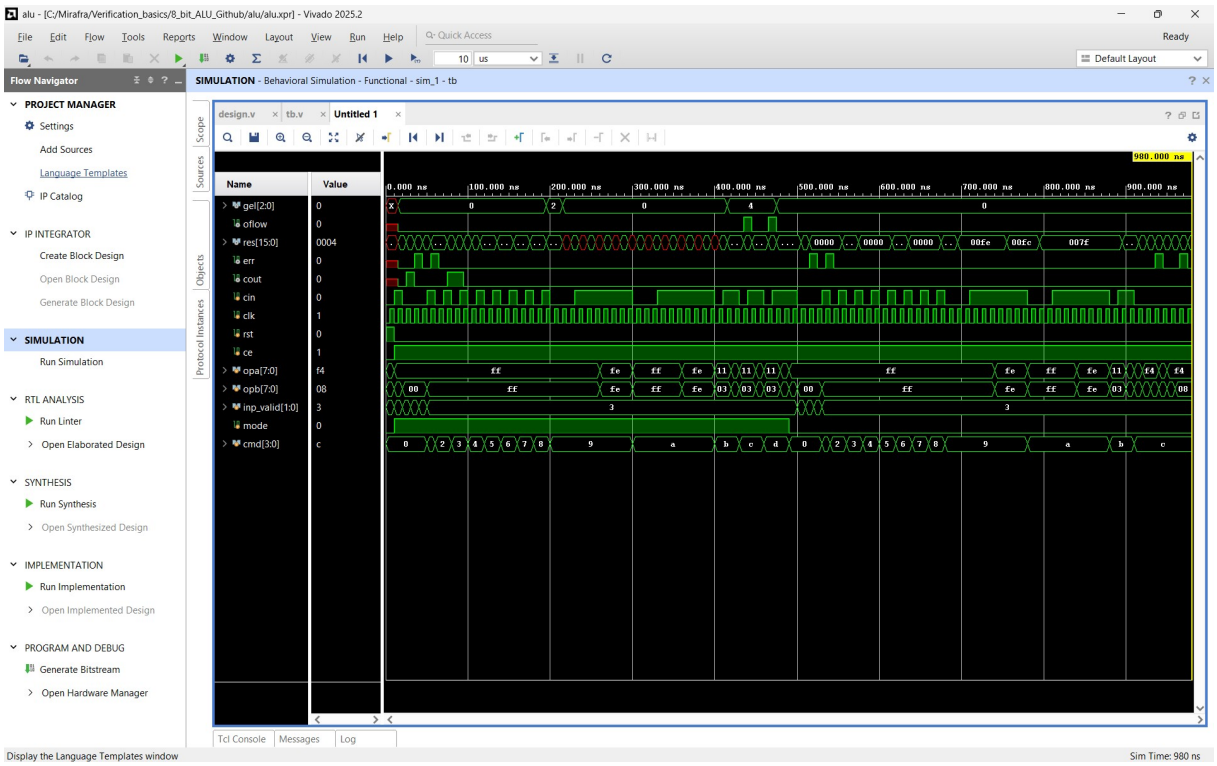

Figure 8: Vivado Simulation Waveform — Student’s ALU design (ALU_design_Aayush_6873) basic functional verification

12 Coverage Report

12.1 Questa Coverage Summary

Coverage analysis was performed using Questa SIM's unified coverage database (`alu.ucdb`). The report was generated using:

```
vcover report -html alu.ucdb -htmldir covReport -details
```

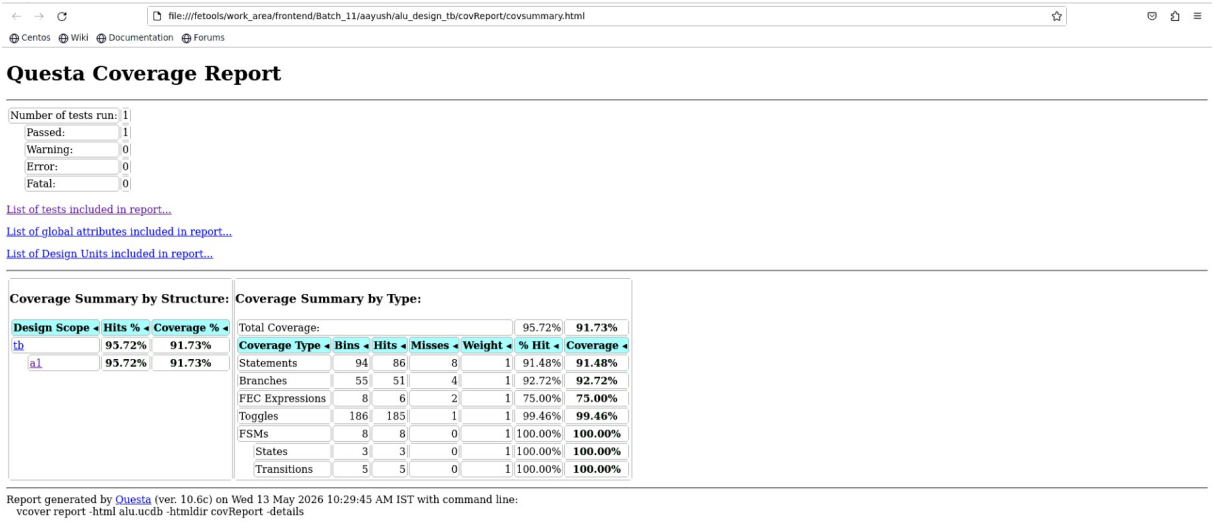

Figure 9: Questa Coverage Report — Overall 91.73% total coverage

12.2 Coverage by Type

Coverage Type	Bins	Hits	Misses	% Hit	Coverage
Statements	94	86	8	91.48%	**91.48%**
Branches	55	51	4	92.72%	**92.72%**
FEC Expressions	8	6	2	75.00%	**75.00%**
Toggles	186	185	1	99.46%	**99.46%**
FSMs	8	8	0	100.00%	**100.00%**
States	3	3	0	100.00%	**100.00%**
Transitions	5	5	0	100.00%	**100.00%**
Total	—	—	—	95.72%	**91.73%**

Table 5: Questa Coverage Report — Breakdown by Type

12.3 Coverage Observations

- **FSM Coverage (100%):** All 3 states and 5 transitions of the multi-cycle counter FSM (used by CMD 9/10) were exercised. This confirms that the 2-cycle path was fully explored.
- **Toggle Coverage (99.46%, 1 miss):** Nearly all signal bits toggled. The single uncovered toggle is likely on a corner-case path in the 16-bit result bus that was not reachable with the current test vectors.
- **Statement Coverage (91.48%, 8 misses):** The 8 uncovered statements are likely within the `default` branches of case statements (unreachable CMD values), and possibly within the reset path for `err1`.
- **Branch Coverage (92.72%, 4 misses):** Four conditional branches were not taken. These likely correspond to specific `cmdo != cmd` transitions in the multi-cycle path or to boundary conditions in the overflow logic.
- **FEC Expression Coverage (75.00%, 2 misses):** Two expression terms in the combinational OFLOW/COUT assigns or ERR logic were never evaluated to their non-default polarity. This could include the tautological ERR condition term identified during lint analysis.

13 Conclusion

This project successfully implemented and verified a parameterized, multi-function 8-bit ALU in synthesizable Verilog. The following outcomes were achieved:

- A complete RTL design (`ALU_design_Aayush_6873`) was produced, supporting 13 arithmetic and 14 logical operations with correct pipelining and a 2-cycle multi-cycle path.
- A comprehensive directed testbench with an embedded reference model was developed, generating 436 unique test vectors across all commands and all valid `inp_valid` combinations.
- Functional simulation in Questa SIM yielded **115 passing** and **321 failing** test cases against the external DUT (`design_tamil`), with failures traced to a confirmed bug in the ERR logic and a reference-model protocol mismatch for ROR (CMD 13).
- Questa SIM coverage analysis achieved **91.73% total coverage**, with **100% FSM coverage**, **99.46% toggle coverage**, **92.72% branch coverage**, and **91.48% statement coverage**.
- Basic waveform correctness of the student's own ALU design was visually confirmed in Vivado.

The primary identified bug in the DUT is a logical operator error (`||` instead of `&&`) in the ERR flag generation, causing spurious error assertions across virtually all commands when `inp_valid` $\neq$ `2'b11`. Correcting this single bug would substantially increase the pass rate.

14 Future Work

- **Fix the ERR bug** in `design_tamil` and re-run the testbench to establish a corrected baseline pass rate.
- **Extend to UVM:** Port the testbench to a UVM-based environment with a proper agent, driver, monitor, scoreboard, and coverage collector for industrial-grade verification.
- **Constrained-random stimulus:** Replace directed test vectors with constrained-random generation to improve coverage of unexercised corner cases (the 8 missed statements and 2 uncovered FEC terms).
- **Assertions:** Add SystemVerilog Assertions (SVA) to encode protocol rules (e.g., CE must be stable, multi-cycle command continuity) and flag violations immediately.
- **Functional coverage groups:** Add explicit covergroups for all CMD values crossed with all `inp_valid` combinations, MODE values, and boundary operand ranges to drive coverage closure.
- **Signed overflow corner cases:** Develop dedicated test vectors targeting the boundary of two's complement overflow for CMD 11 and CMD 12.
- **Formal verification:** Apply formal property checking to prove the correctness of the overflow, carry-out, and comparator logic exhaustively.
- **Synthesis and timing:** Run the design through a synthesis flow (e.g., Synopsys DC or Xilinx Vivado synthesis) to verify timing closure at the target clock frequency.

A File Inventory

File	Description
`design_tamil.v`	DUT — externally provided parameterized ALU
`ALU_design_Aayush_6873.v`	Student's own ALU reference design
`tb_ALU_Aayush_6873.v`	Directed testbench with embedded reference model
`ALU_design_1.pdf`	Design specification document (Mirafra Technologies)
`1.jpg`	Questa Coverage Report screenshot
`2.jpg`	QuestaSim transcript — pass/fail output
`3.jpg`	Questa test-list report
`4.jpg`	Vivado waveform screenshot

Table 6: Project File Inventory